In this chapter we are going to explore a way to use heaps' properties to be (reasonably) sure to have balanced binary trees.

To explain how this works we will introduce *treaps*, a hybrid between trees and heaps; however, we are going to take a different angle on treaps, somehow unusual in the literature on this data structure.

But first, as always, let's start by introducing a problem that we would like to solve.

3.1 Problem: Multi-Indexing

Your family runs a small grocery shop and you'd like to help your parents keeping up with the inventory; hence, to impress your family and show everyone those computer science classes are worth the effort, you embark on the task to design a digital inventory management tool, an archive for stock keeping, with two requirements:

- Be able to (efficiently) search products by name, to update the stock;
- Get, at any time, the product with the lowest items in stock, so that you are able to plan your next order.

Of course, you could just buy an off-the-shelf spreadsheet, but where would the fun be? Moreover, would anybody be really impressed with that? So, here we go, designing an in-memory data structure that can be queried according to two different criteria.

Clearly, real-world scenarios are more complex than this: you can imagine that each product requires a different time to be shipped to you, that some products are ordered from the same vendors (and therefore you might want to group them in the same order